

DA5402 Assignment 1 Report

Karan Kishore
DA25D400

15th February 2026

Introduction

The assignment focusses on creating a manual MLOps system end to end (without sophisticated UX and public deployment though), starting from handling data to training model and deploying it on drifted data. It also emphasizes the phrase, "Change is the only thing that is constant" which I would like to rephrase as "Change towards equilibrium is the only constant" where equilibrium here means the difference between training data error and real-time error. As long as the trained data works well on the production data named `day2_data.csv` in our case, our model is fine. When the change in distribution is large enough, we need to retrain the model. Manually doing this is one example of why manual MLOps is challenging. I discuss many more aspects in the upcoming sections.

1. The Most Difficult Part of Managing Data Versions Manually

The most difficult aspect of managing data versions manually was maintaining consistency and traceability between multiple datasets, scripts, and model artifacts. Let me go through an example/ Initially, I saved the scaled data as version 2. I now have to change the next data version to v3 in `config.yaml`. When I went to the training part, I realized that the predictions were always zero as the data was a scaled one and the input data is an unscaled one. So I had to modify `data_prep.py`, change the logic in `train.py` and also the commands to run api.

Every time the dataset changed slightly, I had to manually update file paths, version names, and metadata references.

Since the update on version number is done manually in config file, I constantly had to remember which dataset version corresponded to which experiment and which version should be used for which model. Small mistakes—such as forgetting to update a version string or overwriting an older file—could silently break reproducibility. I put a check in place to ensure that same version data isn't overwritten. However, this is just a workaround and is not scalable, especially in real-time data which is subject to change on a daily basis by multiple users. Since file naming was the primary tracking mechanism, human error became the biggest risk factor.

Another challenge was ensuring that intermediate transformations were preserved. Once preprocessing scripts were modified, it became unclear whether previously saved datasets were generated with the updated logic or the old one. There is an issue with version name and logic binding. Without automated lineage tracking, reproducibility relied entirely on discipline and documentation.

2. Ensuring the Production Model Matched the Evaluated Model

To ensure that the model in production was the same one evaluated during training, I used explicit version tagging. After training, the model artifact was saved with a clear version identifier (for example, `model_v3.pkl`). The API service was then configured to load that exact file using `config.yaml` using training parameters having the model and data version.

Additionally, I logged evaluation metrics alongside the model version in a structured file. Before deployment, I manually verified that the model file loaded by the API matched the trained artifact using separate parameters under training.

However, this verification process was manual and therefore fragile.

3. Ensuring `train.py` Used the Correct Data Version

The `config.yaml` file was used as the single source of truth for specifying the dataset path and version. The `train.py` script explicitly read from this configuration file rather than hardcoding data paths.

Before every training run, I validated:

- That `config.yaml` pointed to the intended dataset version.
- That the dataset version matched the documented experiment.

This structure reduced accidental mismatches, but it was still vulnerable to human oversight. If the configuration file was not updated correctly after each time `data_prep.py` is used, the whole architecture breaks due to wrong data version being used.

4. Top Three Features Desired in an Automated MLOps Tool

Based on this experience, the three most important features I would want in an automated MLOps tool are:

- 1. Automated Data Lineage Tracking** The ability to automatically record which dataset version produced which model would eliminate ambiguity. This feature would create a clear dependency graph between data, code, and model and also link logic to the data created, instead of manually saying which version of the `data_prep.py` code generated which data.
- 2. Integrated Model Registry** A centralized registry that tracks model versions, metadata, metrics, and deployment status. Also, this would help understand model performance as time progresses. This would ultimately help us see in production at which time the model failed and debug the reason behind it easier in an automated setting.
- 3. Deployment-Version Synchronization** A mechanism that automatically records which model version is currently serving API requests would prevent deployment mismatches. Ideally, the system would refuse to deploy a model without complete metadata registration.

5. The Breakdown in Tracking the Serving Model Version

I realized that I was unsure which model version the API was actively serving. Say I run the inference code through `uvicorn` and then make some changes to data and retrain the model. If `config.yaml` is not changed, the model being used will still be the previous model and not the same as the one with the updated data. This created uncertainty:

- Was the API still using an older trained model?
- Did the new deployment overwrite the previous version?
- Were evaluation metrics aligned with the deployed model?

This confusion highlights the disadvantages of manual tracking. Even though version numbers were used, there was no centralized reference confirming the currently active production model.

6. How an Automated Model Registry Would Have Helped

An automated Model Registry would have simplified Phase B (Training model and ensuring reproducibility) significantly. Instead of manually naming and copying model files, updating `v[x]` to `v[x+1]` where `x` is some version, the workflow would involve:

- Registering the trained model with metadata and evaluation metrics.
- Promoting a specific model version to production status without reliance on the model version.
- Automatically updating the serving endpoint to use the promoted model in case of any data change and retraining on drifted data, for example.

Such a system would also maintain historical versions, enable rollback. Git commits are useful, but hectic as we would need to always commit each time a model is retrained and look at the exact log to get to that checkpoint. So a model registry can provide clarity during deployment to the developer.

Conclusion

Managing MLOps processes manually provided valuable insight into the complexity involved in Modern MLOps tools. It says why they are needed and not just some fancy tools for tech savvy people. While it is technically possible to maintain version control through disciplined naming conventions and configuration files, the process is fragile and heavily dependent on human vigilance and constant scrutiny.